

Logică digitală

-Curs 4-
Reprezentarea funcțiilor
-2024-

Administrativ

- 14:30-16:00, vineri – Tutoriere (activitate extra)
 - Indi Botoc (an II, CTI-Ro)
 - Stefan Jura (an II, CTI-Ro)
 - Sala: A110
 - Frecventa: saptamanal
 - Examen: P1 - 03-07 iunie
 - Examen: P2 - 27, 28, 29 iunie
 - Sala: A101, D1
-

Algebra booleană și logica digitală

- Forma canonică;
 - Forma standard;
 - Aspecte legate de implementarea funcțiilor booleene cu porților logice;
 - Hărți Karnaugh
-

Funcții booleene

- Tabel de adevăr prin care este specificată

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

mintermi

Un **minterm** este o funcție elementară de n variabile notată m_i^n unde n indică numărul de variabile ale funcției iar i este echivalentul zecimal al **mintermului**.

E_z	X_2	X_1	X_0	m_0^3	m_1^3	m_2^3	m_3^3	m_4^3	m_5^3	m_6^3	m_7^3
0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
2	0	1	0	0	0	1	0	0	0	0	0
3	0	1	1	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0	0
5	1	0	1	0	0	0	0	0	1	0	0
6	1	1	0	0	0	0	0	0	0	1	0
7	1	1	1	0	0	0	0	0	0	0	1

mintermi

A	B	C	minterms			
0	0	0	$\overline{A}$	$\overline{B}$	$\overline{C}$	m0
0	0	1	$\overline{A}$	$\overline{B}$	C	m1
0	1	0	$\overline{A}$	B	$\overline{C}$	m2
0	1	1	$\overline{A}$	B	C	m3
1	0	0	A	$\overline{B}$	$\overline{C}$	m4
1	0	1	A	$\overline{B}$	C	m5
1	1	0	A	B	$\overline{C}$	m6
1	1	1	A	B	C	m7

Sumă de mintermi

□ funcția minterm $m_2^3(X_0, X_1, X_2) = \overline{X_2} \cdot X_1 \cdot \overline{X_0}$

are expresia 1 dacă $X_2 = 0$, $X_1 = 1$ și $X_0 = 0$, și valoarea 0 în rest;

□ orice funcție booleană de n variabile poate fi reprezentată ca **sumă logică de funcții minterm**

$$f(X_0, X_1, \dots, X_{n-1}) = \sum_{i \in K} m_i^n$$

Sumă de mintermi (SOP)

$$F = xy + xy'z + x'yz$$

$$F = \sum(3,5,6,7)$$

$$F = m_3^3 + m_5^3 + m_6^3 + m_7^3$$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Formă canonică disjunctivă

$$F = \sum (3, 5, 6, 7)$$

$$F = m_3^3 + m_5^3 + m_6^3 + m_7^3$$

- **forma canonică disjunctivă** a funcției:- termenii produs logic ai funcției conțin **toate** variabilele funcției, între termeni realizându-se operația **SAU** (disjuncție).

Maxterm

- maxterm este o functie elementară de n variabile notate M_i^n unde i este echivalentul zecimal al n -uplului funcției, aplicat în „0”, interpretat ca un număr binar pe n poziții.
- Funcției **maxterm** îi corespunde o expresie de n variabile în formă M_i^n care în urma evaluării pentru toate toate n -uplurile, ia aceeași valoare ca și $M_i^n = m_i^n$.

2.Reprezentarea funcțiilor de comutație

E_z	X_2	X_1	X_0	M_0^3	M_1^3	M_2^3	M_3^3	M_4^3	M_5^3	M_6^3	M_7^3
0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	0	1	1	1	1	1	1
2	0	1	0	1	1	0	1	1	1	1	1
3	0	1	1	1	1	1	0	1	1	1	1
4	1	0	0	1	1	1	1	0	1	1	1
5	1	0	1	1	1	1	1	1	0	1	1
6	1	1	0	1	1	1	1	1	1	0	1
7	1	1	1	1	1	1	1	1	1	1	0

2.Reprezentarea funcțiilor de comutație

□ Funcția maxterm M_3^3 de exemplu are expresia $M_3^3 = X_2 + \overline{X_1} + \overline{X_0} = 0$ pentru $X_2 = 0, X_1 = 1, X_0 = 1$ pentru celelalte atribuiri având valoarea „1”.

□ O funcție de comutație de n variabile poate fi reprezentată printr-un produs de maxtermi:

$$f(X_0, X_1, \dots, X_{n-1}) = \prod_{i \in K_0} M_i^n$$

unde K_0 este mulțimea indicilor M_i^n pt care care funcția ia valoarea „0”.

Maxtermi & Mintermi: $M_i^n = \overline{m_i^n}$

A	B	C	maxterms		minterms		
0	0	0	$A + B + C$	M0	$\overline{A}$	$\overline{B}$	$\overline{C}$ m0
0	0	1	$A + B + \overline{C}$	M1	$\overline{A}$	$\overline{B}$	C m1
0	1	0	$A + \overline{B} + C$	M2	$\overline{A}$	B	$\overline{C}$ m2
0	1	1	$A + \overline{B} + \overline{C}$	M3	$\overline{A}$	B	C m3
1	0	0	$\overline{A} + B + C$	M4	A	$\overline{B}$	$\overline{C}$ m4
1	0	1	$\overline{A} + B + \overline{C}$	M5	A	$\overline{B}$	C m5
1	1	0	$\overline{A} + \overline{B} + C$	M6	A	B	$\overline{C}$ m6
1	1	1	$\overline{A} + \overline{B} + \overline{C}$	M7	A	B	C m7

Produs de maxtermi

$$F = \sum(3, 5, 6, 7)$$

$$F = m_3^3 + m_5^3 + m_6^3 + m_7^3 = \sum(3, 5, 6, 7)$$

$$F = M_0^3 \cdot M_1^3 \cdot M_2^3 \cdot M_4^3 = \prod(0, 1, 2, 4)$$

- **forma canonică conjunctivă** a funcției:- termenii sumă logică ai funcției conțin **toate** variabilele funcției, între termeni realizându-se operația **ȘI**.
-

Forme canonice

- Două forme canonice:
 - Sumă de mintermi
 - Produs de maxtermi
 - Formele canonice sunt unice!
 - Conversia dintr-o formă canonică în alta se face:
 - Interschibând Σ și Π
 - Listând toți indicii lipsă
-

Glossar

Termen	Definiție
Literal	Variabilă booleană sau complementul ei
Termen Produs (Product Term)	Literal sau produs logic ($\&$) între mai mulți literali
Termen Sumă (Sum Term)	Literal sau sumă logică (OR) între mai mulți literali
Sum of Products (SOP)	Sumă logică (OR) între mai mulți termeni produs
Products of Sums (POS)	Produs logic ($\&$) între mai mulți termeni sumă
Minterm	Caz particular de termen produs, care conține toate variabilele de intrare o singură dată
Maxterm	Caz particular de termen sumă, care conține toate variabilele de intrare o singură dată
Sumă de produse canonică	Sumă logică (OR) de acei mintermi aferenți rândurilor din tabelul de adevăr al funcției de ieșire unde aceasta are valoarea 1 logic
Produs de sume canonic	Produs logic ($\&$) de acei mintermi aferenți rândurilor din tabelul de adevăr al funcției de ieșire unde aceasta are valoarea 0 logic

n variabile $\rightarrow 2^{2^n}$ funcții



X	Y	16 possible functions (F_0 – F_{15})															
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
		X AND Y		X		Y		X XOR Y		X OR Y		X NOR Y NOT (X OR Y)		X = Y		NOT Y	
																NOT X	
																X NAND Y NOT (X AND Y)	

NAND



X	Y	Z
0	0	1
0	1	1
1	0	1
1	1	0

$$Z = \overline{X \cdot Y}$$

AND



X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

$$Z = X \cdot Y$$

NOR



X	Y	Z
0	0	1
0	1	0
1	0	0
1	1	0

$$Z = \overline{X + Y}$$

OR



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

$$Z = X + Y$$

Porți logice (cont.)

XOR
 $(X \oplus Y)$



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	0

$Z = X \bar{Y} + \bar{X} Y$
X or Y but not both
("inequality", "difference")

XNOR
 $\overline{(X \oplus Y)}$



X	Y	Z
0	0	1
0	1	0
1	0	0
1	1	1

$Z = \bar{X} \bar{Y} + X Y$
X and Y the same
("equality")

Porți logice

- ❑ Fiecare poartă logică realizează una sau mai multe funcții logice;
 - ❑ Colecția de porți logice folosită în realizarea unui circuit se numește **bibliotecă de porți**, iar porțile din cadrul ei **porți standard**;
 - ❑ Bibliotecile moderne conțin zeci de porți a.î. să scadă costul cu întreținerea și să simplifice realizarea tool-urilor CAD
-

Minimizarea funcțiilor logice

□ se înțelege simplificarea/rescrierea ecuațiilor logice booleene în vederea:

■ Unui cost mai mic și/sau;

■ Performanță mai ridicată;

□ Cheia simplificării este: $y(x + \bar{x}) = y$

■ distributivitatea - $x(y+z) = xy+xz$ —

■ Proprietatea complementului $x + \bar{x} = 1$

Minimizarea funcțiilor logice

- Găsirea a doi termeni (suma sau produs funcție de reprezentarea dorită SOP/POS) pentru care:
 - funcția ia valoare 1
 - numai o variabilă își modifică valoarea

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0

B are aceeași
valoare → B este
păstrat

A are valori
diferite → A este
eliminat

$$F = \bar{A}\bar{B} + A\bar{B} = (\bar{A} + A)\bar{B} = \bar{B}$$

Metoda de minimizare Karnaugh

□ Diagramele Karnaugh:

- Metodă alternativă tabelelor de adevăr și ecuațiilor logice de a vizualiza o funcție
 - se aplică atât pentru ecuațiile logice descrise sub formă canonică de sumă de produse (SOP), cât și pentru ecuațiile logice descrise sub formă canonică de produs de sume (POS)
-

Diagramele Karnaugh

- ❑ constituie o matrice de pătrate cu proprietatea ca două celule vecine corespund unor mintermi **adiacenți**.
 - ❑ doi vectori sunt adiacenți dacă diferă valoric printr-un singur bit
 - ❑ în diagramă se marchează acei mintermi care au valoarea logică 1 în tabelul de adevăr
-

Diagrame Karnaugh

- ❑ Numerele adiacente numărului 0100 sunt: 0101; 0110; 0000; 1100.
 - ❑ Numerele adiacente numărului 000 sunt: 001; 010; 100.
 - ❑ Vectorii adiacenți mintermului abc sunt: $\bar{a}\bar{b}\bar{c}$, $\bar{a}\bar{b}c$, abc
-

Construcție diagrame Karnaugh

- Diagrame Karnaugh pentru funcții logice cu 2 variabile a, b

		b	
		0	1
a	0	(00)	(01)
	1	(10)	(11)

00 - $\bar{a}\bar{b}$

01 - ~~$\bar{a}b$~~ $\bar{a}b$

11 - ab

10 - ~~$a\bar{b}$~~ $a\bar{b}$

Construcție diagrame Karnaugh

- Diagrame Karnaugh pentru funcții logice cu 3 variabile a, b, c

		b			
		bc			
a		00	01	11	10
0		000	001	011	010
1		100	101	111	110

c

Diagrame Karnaugh

□ Ex. completare diagramă:

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		b			
		bc			
a	0	00	01	11	10
	0	000	001	011 1	010
1	1	100	101 1	111 1	110 1

1. Introducerea mintermilor în diagramă conform tabelului de adevăr.

Diagrame Karnaugh

□ Ex. completare diagramă:

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

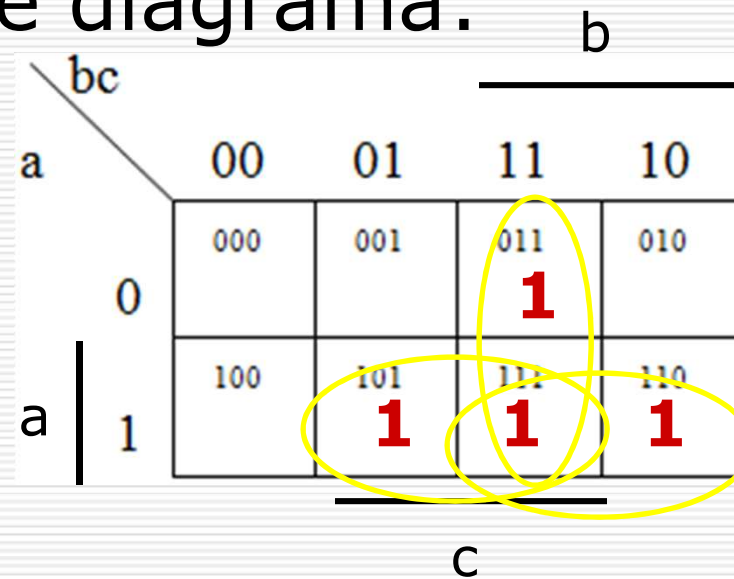
		b			
		bc			
a	0	00	01	11	10
	0	000	001	011 1	010
1	1	100	101 1	111 1	110 1

2. se încearcă formarea unor grupe de mintermi bazate pe reguli de adiacență

Diagrame Karnaugh

□ Ex. completare diagramă:

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



2. O grupare are forma unor dreptunghiuri/pătrate și conține 2^n mintermi!

Diagrame Karnaugh

□ Ex. completare diagramă:

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		b			
		bc			
a	0	00	01	11	10
	0	000	001	011 1	010
a	1	100	101 1	111 1	110 1

2. O grupare are forma unor dreptunghiuri/pătrate și conține 2^n mintermi!

Diagrame Karnaugh

□ Ex. completare diagramă:

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

		b			
		bc			
a	0	00	01	11	10
	0	000	001	011 1	010
a	1	100	101 1	111 1	110 1

2. Din totalul de m variabile booleene a funcției, termenul asociat grupării formate va conține $m-n$ variabile

Diagrame Karnaugh

□ Ex. completare diagramă:

		bc			
		00	01	11	10
a	0	000	001	011 1	010
	1	100	101 1	111 1	110 1

2. Din totalul de m variabile booleene a funcției, termenul asociat grupării formate va conține $m-n$ variabile

Minimizarea folosind diagrame Karnaugh

- Dacă la o astfel de grupare nu mai pot fi adăugați mintermi înseamnă că s-a obținut un **implicant prim**.
 - Dacă un anumit implicant prim conține cel puțin un minterm care nu apare în alt implicant prim atunci acesta este un **implicant prim esențial**
-

Diagrame Karnaugh

□ Ex. completare diagramă:

		b			
		bc			
a	0	000	001	011 1	010
	1	100	101 1	111 1	110 1

$$f = ab + ac + bc$$

3. Ecuația minimizată va conține toți implicantii primi esențiali, și uneori și implicantii primi neesențiali, astfel încât toate celule marcate cu 1 logic să fie acoperite.

Diagramme Karnaugh

$xy \backslash zw$	00	01	11	10
00	⁰ $x'y'z'w'$	¹ $x'y'z'w$	³ $x'y'zw$	² $x'y'zw'$
01	⁴ $x'yz'w'$	⁵ $x'yz'w$	⁷ $x'yzw$	⁶ $x'yzw'$
11	¹² $xyz'w'$	¹³ $xyz'w$	¹⁵ $xyzw$	¹⁴ $xyzw'$
10	⁸ $xy'z'w'$	⁹ $xy'z'w$	¹¹ $xy'zw$	¹⁰ $xy'zw'$

Map Organization

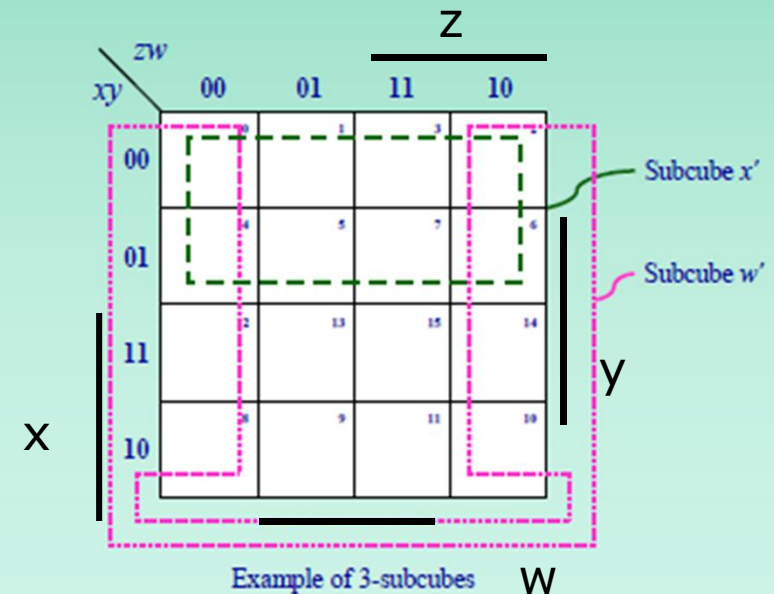
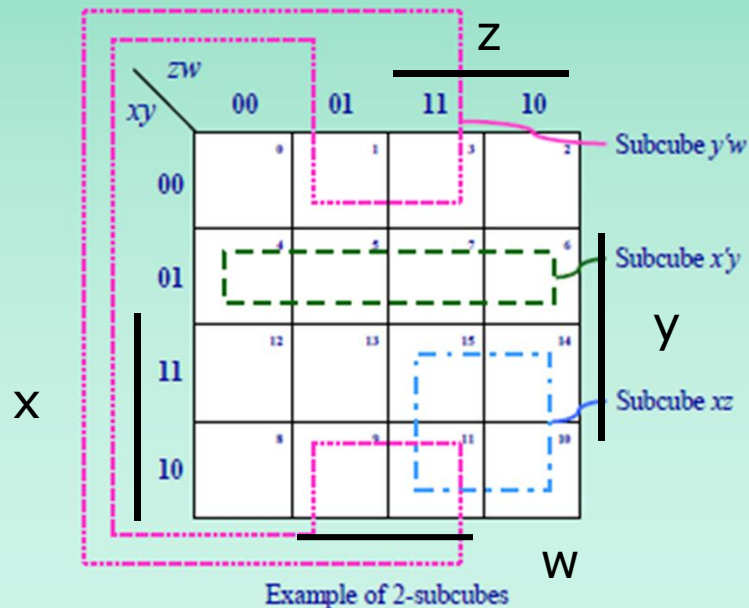


Diagramme Karnaugh

x_1	x_0	y_1	y_0	Greater Than	Equal	Less Than
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

Truth Table

$x_1x_0 \backslash y_1y_0$	00	01	11	10
00	0	1	3	2
01	1	5	7	6
11	1	13	15	1
10	1	9	11	10

Greater-than Function

$$G = x_1y_1' + x_0y_1y_0' + x_1x_0y_0'$$

$x_1x_0 \backslash y_1y_0$	00	01	11	10
00	0	1	1	1
01	4	5	1	6
11	12	13	15	14
10	8	9	1	10

Less-than Function

$$L = x_1'y_1 + x_1'x_0y_0 + x_0y_1y_0$$

Diagrame Karnaugh – don't care

- ❑ Nu toate funcțiile logice sunt definite complet.
 - ❑ Pentru unele valori ale variabile de intrare funcția este nu specificată (funcția are „valoarea” don't care - d)
 - ❑ Pt. "d" în diagrama Karnaugh se va lua în considerare valoarea care ne convine pentru d (0 sau 1) a.î. să permită o acoperire mai largă a minternilor.
-

a	b	c	p	q
0	0	0	0	1
0	0	1	1	0
0	1	0	1	d
0	1	1	d	1
1	0	0	0	d
1	0	1	0	0
1	1	0	d	0
1	1	1	1	1

Tabelul 5.3

Funcțiile p și q pot fi scrise și altfel:

$$p = f(a, b, c) = \sum (1; 2; 7) + \sum d(3; 6)$$

$$q = f(a, b, c) = \sum (0; 3; 7) + \sum d(2; 4)$$

a	bc			
	00	01	11	10
0		1	d	1
1			1	d

a	bc			
	00	01	11	10
0	1		1	d
1	d		1	

$$p = b + \bar{a}c$$

$$q = \bar{b}\bar{c} + bc$$

Porți logice

- ❑ Fiecare poartă logică realizează una sau mai multe funcții logice;
 - ❑ Colecția de porți logice folosită în realizarea unui circuit se numește **bibliotecă de porți**, iar porțile din cadrul ei **porți standard**;
 - ❑ Bibliotecile moderne conțin zeci de porți a.î. să scadă costul cu mentenanța și să simplifice realizarea tool-urilor CAD
-

Porți logice

- Criteriile pt. selecția operatorilor:
 - Frecvența utilizării porții;
 - Extensibilitatea operatorului pentru mai mult de 2 variabile;
 - Costul prin prisma nr. de tranzistori, respectiv timpul de comutare al porții;
 - 8 operatori sunt selectați: NOT(Inverter), Driver (comandarea multor circuite (load mare), linii lungi), And, Or, NOR, NAND, XOR, XNOR
-

PORTI LOGICHE ELEMENTARE

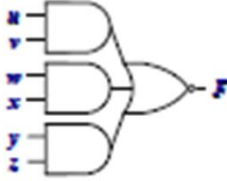
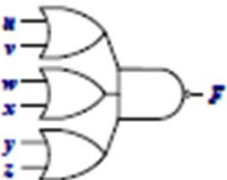
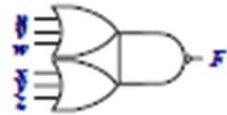
Name	Graphic Symbol	Functional Expression	Number of transistors	Delay in ns
Inverter		$F = x'$	2	1
Driver		$F = x$	4	2
AND		$F = xy$	6	2.4
OR		$F = x + y$	6	2.4
NAND		$F = (xy)'$	4	1.4
NOR		$F = (x + y)'$	4	1.4
XOR		$F = x \oplus y$	14	4.2
XNOR		$F = x \odot y$	12	3.2

Porți logice

- ❑ Modul prin care pot fi optimizate:
 - Întârzierea (delay)
 - Costul (nr.tranzistori)
 - ❑ 2 posibilități:
 - Porți cu mai multe intrări (operatori asociativi și comutativi). Ex.: AND, OR
 - Porți complexe:
 - ❑ mai multe intrări
 - ❑ Realizează operații multiple
-

PORȚI LOGICE CU MAI MULTE INTRĂRI

Name	Graphic Symbol	Functional Expression	Number of transistors	Delay in <i>ns</i>
3-input AND		$F = xyz$	8	2.8
4-input AND		$F = xyzw$	10	3.2
3-input OR		$F = x + y + z$	8	2.8
4-input OR		$F = x + y + z + w$	10	3.2
3-input NAND		$F = (xyz)'$	6	1.8
4-input NAND		$F = (xyzw)'$	8	2.2
3-input NOR		$F = (x + y + z)'$	6	1.8
4-input NOR		$F = (x + y + z + w)'$	8	2.2

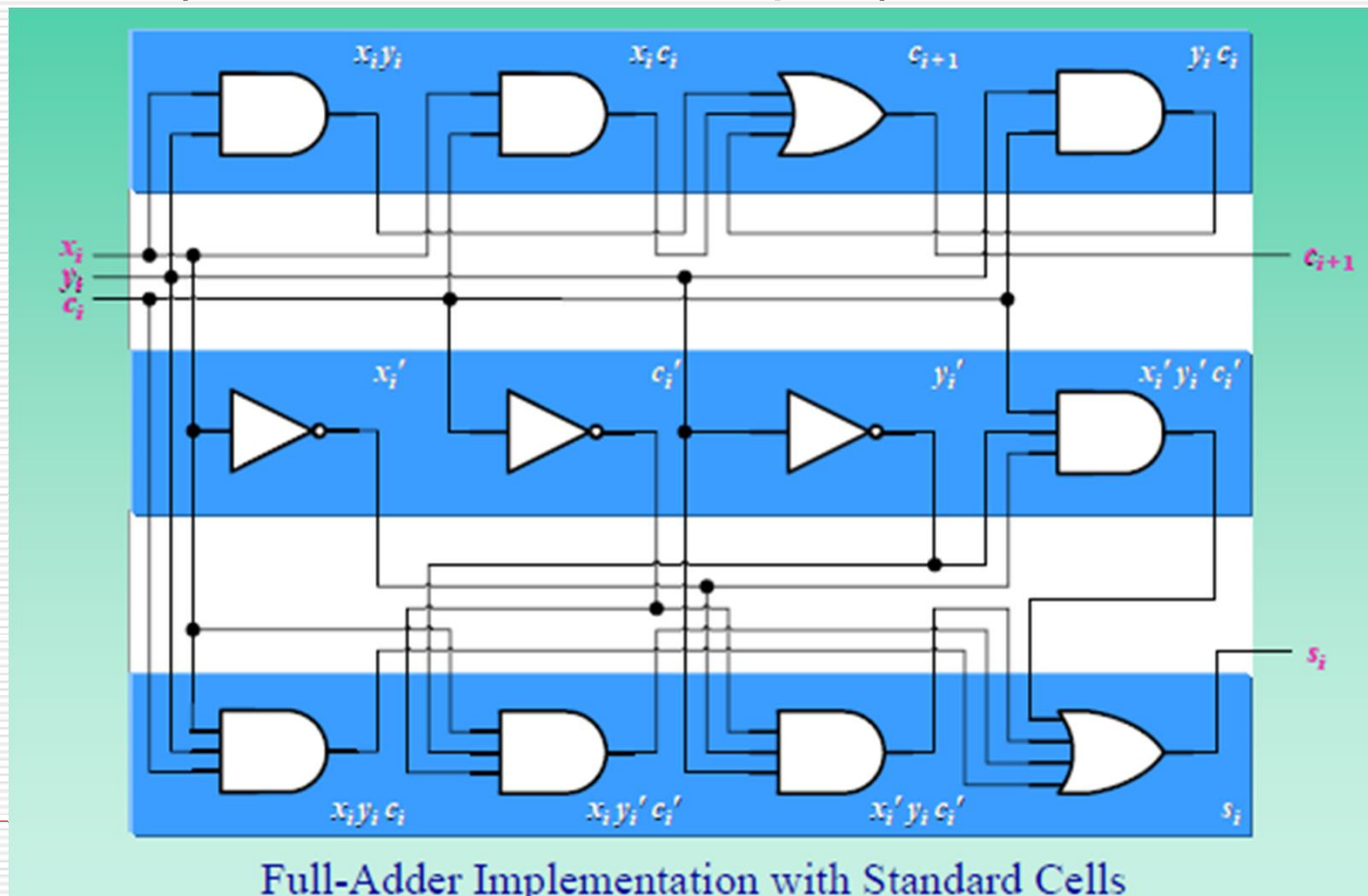
Name	Graphic Symbol	Functional Expression	Number of transistors	Delay in ns
2-wide, 2-input AOI		$F = (wx + yz)'$	8	2.0
3-wide, 2-input AOI		$F = (uv + wx + yz)'$	12	2.4
2-wide, 3-input AOI		$F = (uvw + xyz)'$	12	2.2
2-wide, 2-input OAI		$F = ((w + x)(y + z))'$	8	2.0
3-wide, 2-input OAI		$F = ((u + v)(w + x)(y + z))'$	12	2.2
2-wide, 3-input OAI		$F = ((u + v + w)(x + y + z))'$	12	2.4

Tehnologia VLSI

- ☐ **Small-scale integration (SSI)**
 - 10 gates/package
 - ☐ **Medium-scale integration (MSI)**
 - 10 – 100 gates/package (2 – 4 bit slices)
 - ☐ **Large-scale integration (LSI)**
 - 100 – 1000 gates/package (controllers, datapaths, bit slices)
 - ☐ **Very-large-scale integration (VLSI)**
 - 1000+ gates/package (systems on a chip)
 - ☐ ***Custom designs (Standard cells)***
 - ☐ ***Gate arrays (GAs)***
 - ☐ ***Field-programmable (FPGAs)***
-

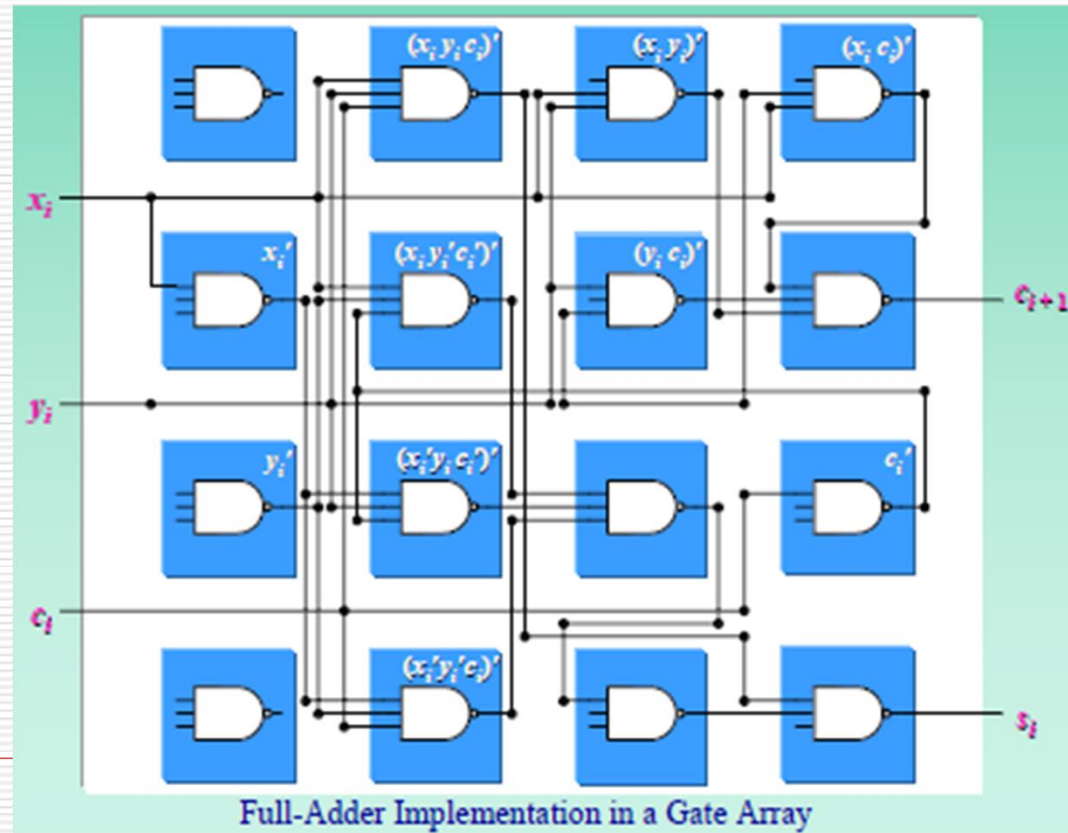
Custom design

- ❑ Folosește o bibliotecă de porți standard;



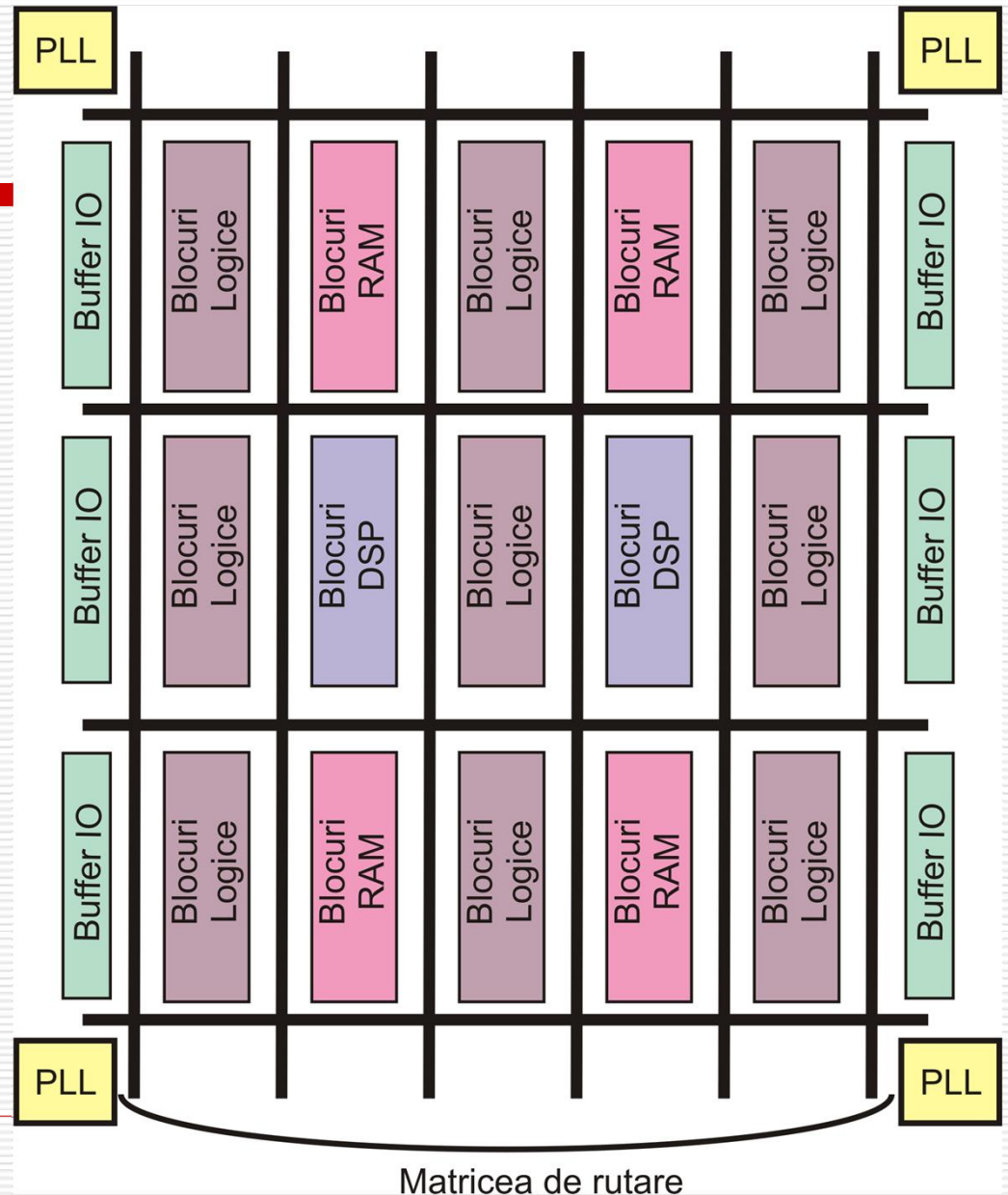
Semi-custom approach

- Gate array-uri: aceeași poartă, interconecturile sunt partea custom



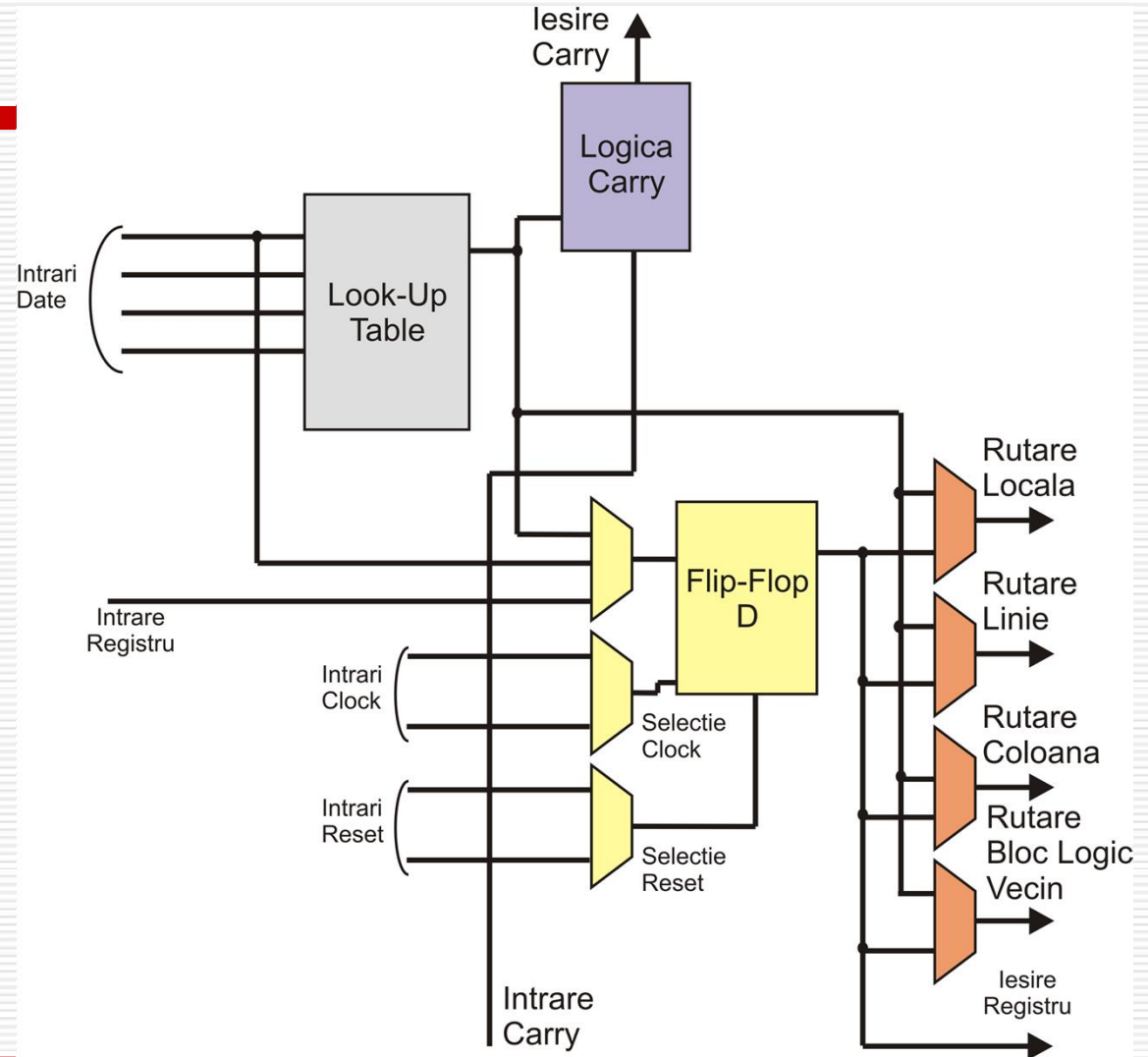
FPGAs

- ❑ **F**ield
Programable
Gate **A**rrays
(FPGA)
- ❑ Sunt programate prin încărcarea fișierului *.bit/*.sof în memoria internă;
- ❑ Sunt folosite pentru prototipaj;



FPGAs

- ❑ Bloc logic - **Look-up table (LUT)** - folosit la implementarea funcțiilor logice combinaționale;
- ❑ un LUT - **3-6 intrări de date** (în funcție de tipul FPGA-ului) și o ieșire;
- ❑ până la **65536 de funcții logice** combinaționale de **3-6 variabile**

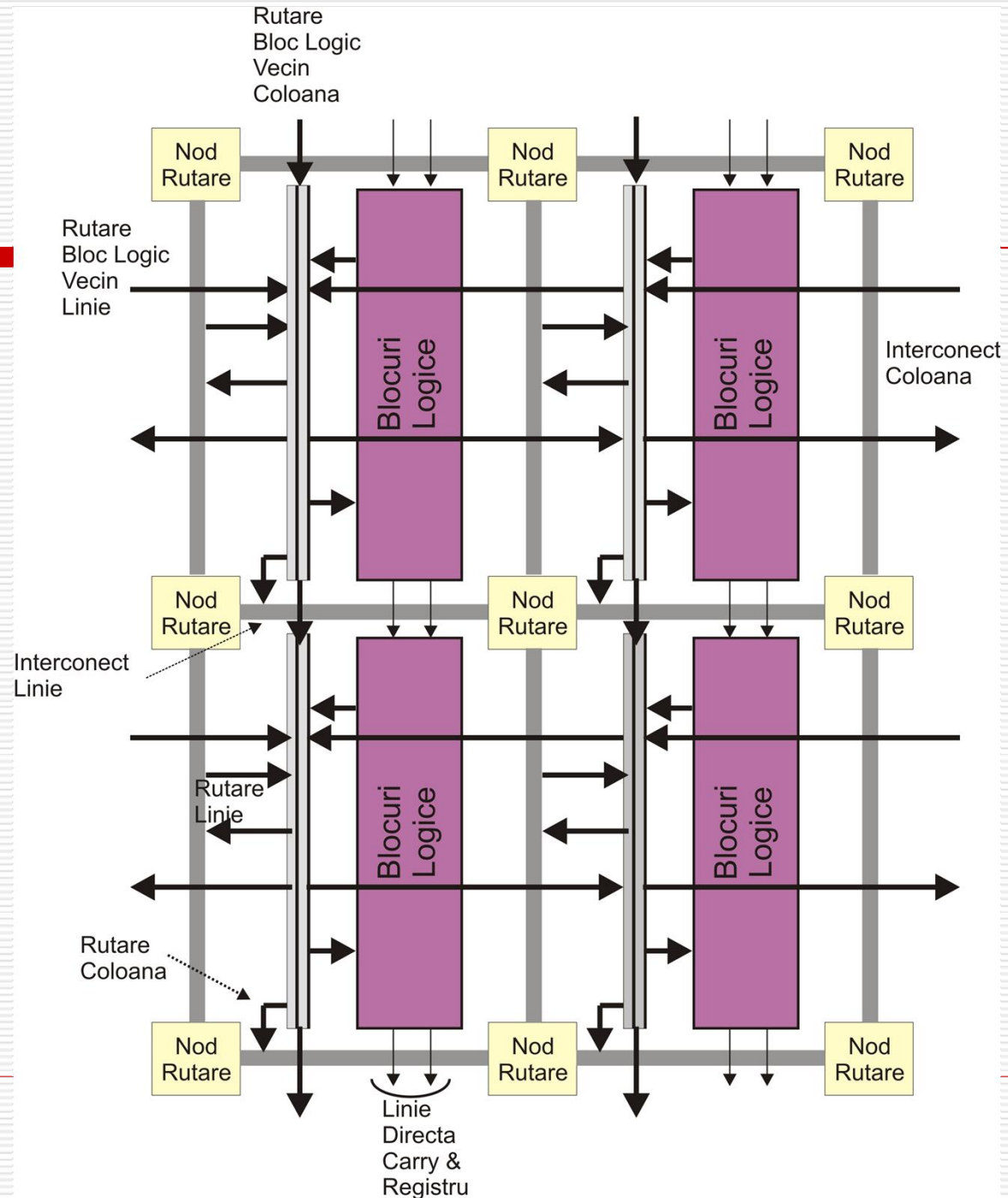


FPGAs

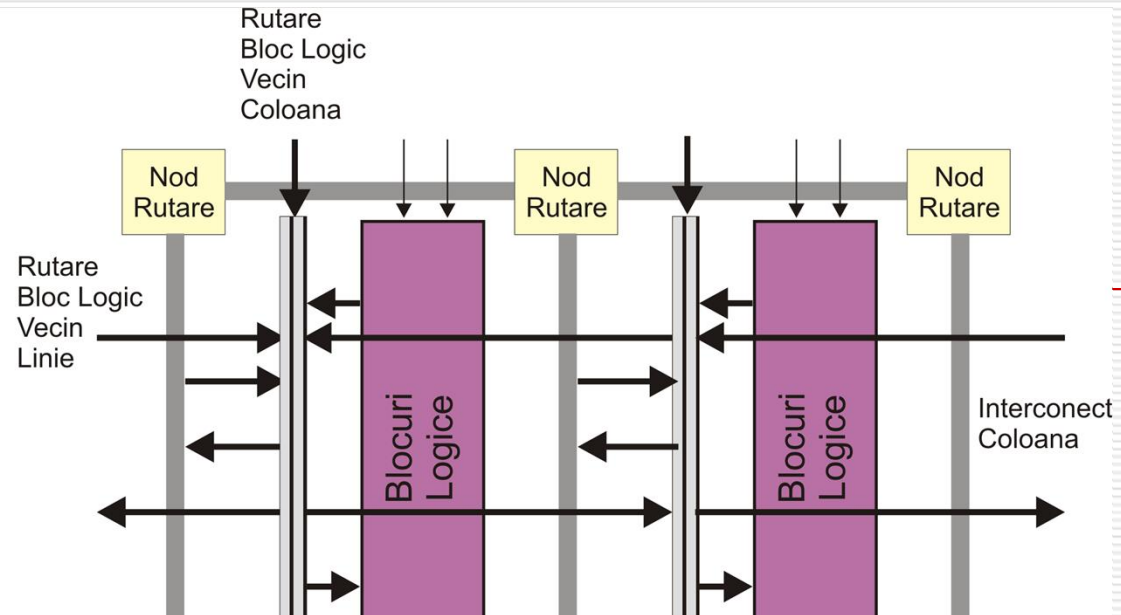
Rutare:

- linie
- coloană
- în interiorul blocului

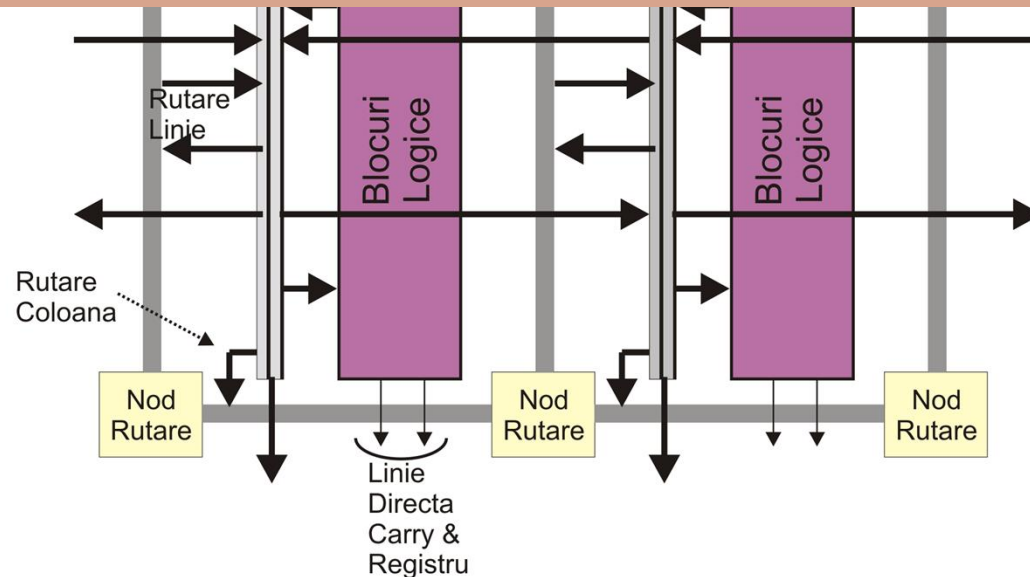
Interconexiuni
între blocurile
alăturate pe
linie și pe
coloană.



FPGAs



Rutarea la nivel de linie, respectiv coloană, introduce întârzieri **importante**, de multe ori mai mari decât cele introduse datorită logicii, pentru circuitele implementate în FPGA!



Întrebări?
